Architecture Document — v2

De-Platformed Mesh Messenger — Pairing Model & Extensible Transport (BLE / Wi-Fi Direct / LoRa / Ham Radio)

Project:	De-Platformed Mesh Messenger & Local Network Bridge
Prepared By:	_____
Date:	_____
Version:	2.4 — Competitive Differentiation and Help Request/Location Sharing added; Hybrid/Cloud mode moved to future roadmap, not current scope

1. What This Revision Changes

- Adds an explicit Pairing / Trust layer: two users must exchange and accept a connect request before a chat exists between them.
- Formalizes that background relay (store-and-forward) requires no pairing and no per-message permission — enforced cryptographically, not just by policy.
- Introduces a Transport Abstraction so the mesh can run over Bluetooth/Wi-Fi Direct today and LoRa or Ham radio hardware in future, without changing the layers above it.
- Adds a Security / Threat Model covering malicious content, OS-level radio stack risks, and malicious relay behavior.
- Adds Group Chat / Rooms: multi-member conversations built on top of the pairing model, using a shared rotatable sender key.
- Adds Admin Broadcast Posts: signed posts from designated admin identities, flood-propagated to the whole connected mesh, verifiable offline.
- Documents Hybrid Online/Offline Mode as a future roadmap item (Section 12) — a cloud relay path that could be added later, with the mesh as the fallback. Not part of the current build, which is mesh-only over Bluetooth and Wi-Fi Direct.
- Adds Competitive Differentiation and a Help Request / Location Sharing feature (Sections 13-14).

2. System Architecture — 5 Layers

Layer	Responsibility	Permission Required?
5. UI / App	Chat, bulletin board, resource map, mesh status	N/A
4. Sync Engine (CRDT)	Merge offline data from any device, any order, no server	N/A
3. Pairing / Trust (NEW)	Connect requests, accept/reject, key exchange between two specific users	Yes — explicit, one-time per contact
2. Security	End-to-end encryption of all payloads before transport	N/A — enforced by keys from Layer 3
1. Transport (Abstracted)	Device discovery, connection, store-and-forward hopping over any radio	No — relay is permissionless by design

The critical design rule: pairing (Layer 3) governs who can appear in your chat list and read your messages. It has no effect on Layer 1 — every device, paired or not, still relays encrypted traffic for the whole mesh. These are independent systems that happen to share the same wire.

3. Layer 3 — Pairing / Trust (New)

3.1 Flow

• User A sends a connect request to User B (travels over Layer 1, same as any message — direct if in range, relayed through the mesh if not).
• User B sees the request in-app and explicitly accepts or rejects it.
• On accept, both devices run CryptoManager.performX3DH() to derive a shared session key.
• Only after this exchange does a chat thread exist between A and B — nothing is decryptable before this point.
• Either party can later revoke/block, which deletes the local session key and ends the ability to read new messages from that peer.

3.2 Core Functions / Module: ContactManager

```
ContactManager.sendConnectRequest(peerId) -> requestId
ContactManager.getPendingRequests() -> [requestId]
ContactManager.acceptRequest(requestId) -> triggers CryptoManager.performX3DH(peerBundle)
ContactManager.rejectRequest(requestId)
ContactManager.isPaired(peerId) -> bool
ContactManager.revokePairing(peerId) -> deletes local session key
ChatUI.listConversations() -> only peers where isPaired(peerId) == true
```

3.3 Why Relay Cannot Be Spoofed by an Unpaired Device

A relay device forwards an encrypted envelope it cannot decrypt, because it never performed X3DH with either party and has no session key. The Double Ratchet on the receiving end will reject any payload not authenticated with the real shared secret. This means the 'carry only, cannot send' rule is enforced by the cryptography itself, not by an app-level permission check that could be bypassed.

4. Layer 1 — Transport Abstraction (Current + Future Radios)

All relay logic is written once against a generic TransportProvider interface. Any radio — BLE, Wi-Fi Direct, LoRa, or Ham radio — implements the same interface, so adding a new radio never requires touching the pairing, encryption, or sync layers above it.

4.1 TransportProvider Interface

```
TransportProvider.discover(onPeerFound)
TransportProvider.connect(peerId)
TransportProvider.send(peerId, bytes)
TransportProvider.onReceive(callback)
TransportProvider.disconnect(peerId)
TransportProvider.getRangeEstimate() -> meters
```

4.2 Transport Implementations

Implementation	Status	Range	Requires Extra Hardware?
BLETransport	Current (MVP)	~10-100m per hop	No — built into every phone

WiFiDirectTransport	Current (MVP, Android)	~100-200m per hop	No — built into every phone
LoRaTransport	Future	~2-15km per hop	Yes — small LoRa radio module (e.g. Meshtastic-class device) paired to phone via Bluetooth
HamRadioTransport	Future	5-50km+, further with repeaters	Yes — ham radio handheld/module; operator needs a license
CloudTransport (Sec. 12)	Future — Hybrid mode	Unlimited, wherever internet reaches	No — uses the phone's existing internet connection when present

4.3 Why LoRa and Ham Radio Are Future, Not MVP

- Both require hardware the user doesn't already own — this is a real cost and adoption barrier, unlike BLE/Wi-Fi Direct which need nothing extra.
- Ham radio additionally requires the operator to hold a license in most countries — a legal constraint, not a technical one.
- The payoff: once added, the SAME app, pairing model, and encryption work unchanged over these radios — only the transport implementation is new.
- Recommended future path: follow the Meshtastic model — a cheap LoRa dongle communicates with the phone over Bluetooth locally, while the dongle-to-dongle link uses LoRa for long range.

5. End-to-End Message Flow (Updated)

Step	What Happens	Layer
1	User A and B exchange connect requests and accept each other.	3 — Pairing
2	X3DH runs, deriving a shared session key on both devices.	3 — Pairing / 2 - Security
3	User A writes a message; CryptoManager encrypts it with the session's Double Ratchet key.	2 — Security
4	SyncEngine wraps it as a CRDT change and hands it to the outbox.	4 — Sync
5	RelayEngine sends it via whichever TransportProvider(s) are active (BLE, Wi-Fi Direct, or future LoRa/Ham).	1 — Transport
6	Any intermediate device forwards the still-encrypted envelope without pairing or decrypting it.	1 — Transport
7	User B's device receives it, decrypts with the shared session key, and merges it into the CRDT thread.	2/4
8	If B is offline, any device with connectivity bridges the queue to the backend/internet.	1 — Transport (Bridge)

6. Updated / New Functional Requirements

ID	Requirement	Layer	Priority
FR-11	Users shall send and receive connect requests before a chat thread exists.	3	High
FR-12	System shall run X3DH key exchange automatically upon request acceptance.	3	High
FR-13	System shall allow revoking a pairing, deleting the local session key.	3	Medium
FR-14	System shall relay any encrypted envelope regardless of the relaying device's pairing status.	1	High
FR-15	System shall reject/ignore payloads that fail Double Ratchet authentication.	2	High
FR-16	Transport layer shall be implemented behind a common interface supporting multiple radio backends.	1	High
FR-17 (Future)	System shall support a LoRa transport implementation for long-range relay via paired hardware.	1	Low — future phase
FR-18 (Future)	System shall support a Ham radio transport implementation with license-aware setup flow.	1	Low — future phase

7. Updated Technology Stack

Layer	Choice	Notes
Pairing / Trust	Custom ContactManager module	Built on top of libsignal's X3DH; no new library needed
Transport — current	Android Nearby Connections API, iOS Multipeer Connectivity	Unchanged from v1
Transport — future (LoRa)	Meshtastic firmware/protocol, or custom LoRa driver over a module (e.g. RFM95, SX1262)	Requires external hardware dongle
Transport — future (Ham)	AX.25 packet radio protocol, or APRS-style framing over a ham handheld/TNC	Requires licensed operator
Encryption	libsignal (Signal Protocol) — unchanged	X3DH + Double Ratchet
Sync engine	Automerge or Yjs — unchanged	CRDT-based, no server required

8. Additional Risks From This Revision

Risk	Impact	Mitigation
Users expect chat access before completing pairing (UX confusion)	Medium	Clear in-app state: 'Request sent — waiting for accept' before any chat UI appears

LoRa/Ham hardware adds cost and complexity later	Medium	Keep hardware transports fully optional and behind the same TransportProvider interface; ship without them first
Ham radio legal requirements vary by country	Medium	Treat as a clearly-labeled advanced/optional mode with licensing guidance, not a default

9. Security / Threat Model

Because this app deliberately accepts data over the air from devices it has never met or vetted, its attack surface is broader than a typical app that only talks to a trusted server. Three distinct risk categories apply, each requiring a different mitigation.

9.1 Malicious Message Content

A paired contact, or a relay carrying a message toward you, could deliver a malicious file, link, or crafted payload. This risk is shared with any messaging app and is not unique to mesh transport.

- No auto-open or auto-preview of attachments; require explicit user action to open any file.
- Sandbox all file/media rendering (images, documents) so a malformed file cannot execute code.
- Strip and validate file metadata before display.
- The pairing model (Section 3) already limits who can message you directly — an unpaired device can relay data but cannot open a chat with you.

9.2 Attacks on the Bluetooth / Wi-Fi Stack Itself

Some vulnerabilities live in the phone OS's own radio stack, not in this app's code, and can trigger without pairing or any user action. Precedent: BlueBorne (2017), a set of Bluetooth stack bugs that allowed device takeover from proximity alone, with no pairing required.

- Use only official OS-level radio APIs (Nearby Connections, Multipeer Connectivity) rather than custom raw BLE parsing, so the app inherits the OS vendor's own security patches.
- Request the minimum radio permissions needed; do not hold the radio in broad/promiscuous modes longer than necessary.
- Document this as a residual risk: OS-level radio bugs cannot be fully mitigated from the app layer and depend on the user's device being patched/updated.

9.3 Malicious Relay Devices Tampering With the Protocol

A relay device could attempt to inject fake messages, replay old ones, or send malformed protocol data, since by design the app accepts traffic from devices it has never vetted.

- A relay cannot inject or forge messages into a paired chat: the Double Ratchet on the receiving end rejects anything not authenticated with the real session key (Section 3.3).
- A relay cannot read message content: it never completes X3DH and has no session key.
- All payload/envelope parsing from untrusted or unpaired devices must be fuzz-tested and hardened against malformed or oversized input, since a parser crash or overflow bug here is directly reachable by any nearby device, paired or not.

9.4 New Non-Functional Requirements

ID	Requirement	Category
NFR-07	All payload parsing from untrusted/unpaired devices shall be fuzz-tested and hardened against malformed input.	Security
NFR-08	Attachments/media shall never auto-open or auto-render without explicit user action.	Security

NFR-09	App shall request only the minimum radio permissions required and avoid holding radios in broad scan modes unnecessarily.	Security
NFR-10	App shall rely on official OS radio APIs rather than custom low-level radio parsing.	Security

10. Group Chat / Rooms

Extends the pairing model beyond 1:1 chats. A room is a named group with an explicit member list, created by any user, that others join by invite. Group messages use a shared 'sender key' rather than one pairwise key per pair, so the app doesn't have to encrypt each message once per member.

10.1 Flow

- User A creates a room and becomes its first member.
- User A invites User B (must already be a paired contact, or pairing happens first — Section 3).
- A generates a group sender key and delivers it to B privately, encrypted with their existing 1:1 session key.
- B can now decrypt group messages from anyone in the room who has that sender key.
- When a member is removed, the sender key is rotated and redistributed to remaining members only, so the removed member cannot read future messages (forward secrecy for group membership changes).

10.2 Core Functions / Module: RoomManager

```
RoomManager.createRoom(roomName) -> roomId
RoomManager.inviteMember(roomId, peerId)  // requires isPaired(peerId) == true first
RoomManager.acceptInvite(roomId)
RoomManager.removeMember(roomId, peerId) -> triggers GroupCrypto.rotateSenderKey(roomId)
RoomManager.listRooms() -> [roomId]
RoomManager.sendGroupMessage(roomId, plaintext) -> GroupCrypto.encryptWithSenderKey(roomId, plaintext)
GroupCrypto.generateSenderKey(roomId)
GroupCrypto.distributeSenderKey(roomId, peerId) -> sent via existing 1:1 session
GroupCrypto.decryptGroupMessage(roomId, ciphertext, senderId)
```

10.3 Data Type

Type	Structure
Room	CRDT map: roomId, roomName, memberList, createdBy, createdAt, senderKeyVersion
Group message thread	Append-only log per roomId, same structure as a 1:1 thread but keyed to the room's sender key

11. Admin Broadcast Posts

A small, explicitly designated set of admin identities can publish a post intended to reach every app user in the mesh — distinct from the community bulletin board, where any user can post to their local area. Admin posts need authenticity (proof it really came from an admin) without needing a central server to issue that proof.

11.1 How Authenticity Works Without a Server

- Each admin identity has a public/private signing key pair. Admin public keys are shipped inside the app itself (or added later through a signed app update), so any device can verify a post's signature offline.
- An admin post is signed, not encrypted — its content is meant to be readable by everyone, but the signature proves it wasn't forged or altered in transit.
- The UI shows a verified badge only when the signature check passes against a known admin public key.

11.2 How It Reaches 'All' Users

Admin posts use flood propagation, not the pairwise relay used for private messages: every device that receives a valid, not-yet-seen post forwards it to every neighbor it can reach, deduplicated by post ID to prevent infinite loops. This is the same mechanism as the bulletin board, applied mesh-wide instead of locally. This reaches everyone within the currently connected mesh, plus anyone reachable once a bridge device carries it out to a separately connected part of the mesh — not a guaranteed instant global broadcast if the mesh is fragmented into disconnected pockets.

11.3 Core Functions / Module: BroadcastManager

```
BroadcastManager.publishPost(adminPrivateKey, content) -> signedPost
BroadcastManager.verifyPost(signedPost) -> bool  // checks against known admin public keys
BroadcastManager.floodToMesh(signedPost)  // relay to all neighbors, dedup by postId
BroadcastManager.listRecentPosts() -> [signedPost]
AdminKeyRegistry.getKnownAdminKeys() -> [publicKey]  // shipped with app, updatable via signed app upda
te
```

11.4 New / Updated Functional Requirements

ID	Requirement	Layer	Priority
FR-19	Users shall be able to create a room and invite paired contacts to it.	3/5	Medium
FR-20	Group messages shall use a shared, rotatable sender key rather than per-pair encryption.	2/3	Medium
FR-21	Removing a room member shall rotate the sender key so they cannot read future messages.	2/3	Medium
FR-22	A designated admin identity shall be able to publish a signed broadcast post readable by all users.	1/2	High
FR-23	All devices shall verify a broadcast post's signature before displaying it or its verified badge.	2	High
FR-24	Broadcast posts shall propagate via flood relay to the full connected mesh, deduplicated by post ID.	1	High

12. Hybrid Online / Offline Mode (Future — Not in Current Build)

Status: future roadmap, not part of the current build. The current app is mesh-only — Bluetooth and Wi-Fi Direct, no server, no cloud dependency. This section documents how a cloud/hybrid path could be added later, so the design decision is captured now without committing any current work to building or maintaining a backend.

Internet is only unavailable some of the time — most users have it most days. If built later, the app would use it when present, for speed, reliability, and lower battery cost, and fall back to the mesh transparently the moment it isn't. This would be a natural extension of the layered design already in place: Security (Layer 2) encrypts before handing data to Transport (Layer 1) and never knows or cares which radio carries it. Going hybrid, whenever it happens, would just mean adding one more TransportProvider implementation — a cloud/internet path — alongside BLE and Wi-Fi Direct.

12.1 Design Principle (If Built): Same Message, Different Roads

- A message would be encrypted exactly once (Layer 2), regardless of which path carries it.
- A ConnectivityManager would decide, per send attempt, which TransportProvider to use — cloud first if reachable, mesh as automatic fallback.
- If both paths happened to deliver the same message (e.g. connectivity flickers mid-send), the CRDT sync engine already deduplicates by message ID — no special-case handling would be needed there.

12.2 New Component (If Built): Lightweight Backend Relay

A minimal server would relay encrypted envelopes and store them only until delivered — it would never have the keys to decrypt anything, so it would function the same as the mesh's own relay devices: a blind courier, just a faster and more reliable one when reachable. This mirrors how Signal's own server works.

- Would store encrypted, undelivered messages until the recipient's device comes online and fetches them.
- Would send a push notification (via FCM on Android / APNs on iOS) to wake the recipient's app when a new message is waiting.
- Would host a public directory of pre-key bundles, so two users who have never been physically near each other could still complete X3DH pairing over the internet, instead of requiring in-person proximity.
- Could propagate admin broadcast posts (Section 11) near-instantly to every online user, with mesh flood relay handling only the users who are currently offline.

12.3 Core Functions / Module (If Built): ConnectivityManager + CloudTransport

```
ConnectivityManager.getBestAvailablePath() -> 'cloud' | 'mesh'
ConnectivityManager.onConnectivityChange(callback)
CloudTransport.send(peerId, encryptedBytes)  // implements TransportProvider
CloudTransport.onReceive(callback)
CloudTransport.fetchPreKeyBundle(peerId) -> bundle  // enables remote pairing, no proximity needed
CloudTransport.registerPushToken(deviceToken)
RelayEngine.send(peerId, bytes) {
  path = ConnectivityManager.getBestAvailablePath()
  path == 'cloud' ? CloudTransport.send(...) : MeshRelay.send(...)
}
```

12.4 What Would Change for Each Existing Feature, If This Phase Is Built

Feature	Online Behavior	Offline Behavior
Pairing (Sec. 3)	Instant, works with anyone globally via server pre-key directory	Requires physical mesh proximity, as originally designed

1:1 / group messages	Delivered via backend relay, near-instant	Store-and-forward mesh hopping, as originally designed
Admin broadcast (Sec. 11)	Server pushes to all online users almost immediately	Flood relay across the connected mesh, as originally designed
Bulletin board / map pins	Syncs to backend so it's visible to online users beyond mesh range too	Local-mesh-only visibility, as originally designed

12.5 New / Updated Requirements (Future — Not Current Scope)

ID	Requirement	Priority
FR-25 (Future)	App shall automatically prefer an available internet connection over mesh for sending messages.	Deferred
FR-26 (Future)	App shall fall back to mesh transport transparently when internet is unavailable, with no user action required.	Deferred
FR-27 (Future)	Backend relay shall store only encrypted envelopes and shall never possess decryption keys.	Deferred
FR-28 (Future)	Users shall be able to pair with a non-nearby user via the server's pre-key directory when online.	Deferred
NFR-11 (Future)	Switching between cloud and mesh transport shall not duplicate delivered messages (enforced via CRDT message-ID dedup).	Deferred

12.6 Backend Infra Needed (Only If This Phase Is Built)

- WebSocket or HTTPS relay server (stores/forwards encrypted envelopes only)
- Small database for undelivered message queue and pre-key bundle directory
- Push notification integration: Firebase Cloud Messaging (Android), Apple Push Notification service (iOS)
- None of this is required for the current build to function — the current mesh-only app has a complete, working offline path without any of it. This infrastructure is only needed if and when the team decides to build this phase.

13. Competitive Differentiation

Bridgefy, Briar, and Bitchat are active, real apps solving a similar core problem. This section captures where this app is designed to go further, based on gaps in what they currently offer.

Feature	Bridgefy / Briar / Bitchat	This App
Verified public broadcasts	Public broadcast exists (Bridgefy) but is not cryptographically signed/verifiable	Signed, offline-verifiable admin broadcasts (Sec. 11)
Resource mapping	Not offered by any of the three	Offline resource map with crowd-verified pin status
Personal help/check-in signal	Not offered by any of the three	User-triggered 'I'm safe' / 'need help' broadcast (Sec. 14)
Hybrid online/offline with remote pairing	Briar bridges via Tor; no seamless cloud-first/mesh-fallback with remote pairing in any of the three	Automatic cloud-first, mesh-fallback with remote pairing (Sec. 12)
Battery-aware relay	Common complaint across all three; not addressed as a feature	Relay participation throttles automatically below a battery threshold
Long-range hardware roadmap	None of the three integrate LoRa or ham radio	LoRa / Ham radio as future TransportProvider implementations (Sec. 4)

14. Help Request & Location Sharing

A user-triggered signal for 'I'm safe' or 'I need help,' optionally carrying location. Location is never shared passively or continuously in the background — it is captured only at the moment the user explicitly triggers this feature, and stops the moment it's cancelled or expires.

14.1 Two Tiers, Two Different Privacy Models

This mirrors the same pairing-vs-broadcast distinction already in the architecture: full detail only to people you trust, reduced detail to everyone else.

	Tier 1 — Notify My Circle	Tier 2 — Broadcast to Nearby Responders
Who sees it	Only your paired emergency contacts / room	Anyone in the connected mesh
Location precision	Exact coordinates	Rounded to an approximate grid (e.g. nearest ~100m)
Identity shown	Your name (they already know you)	Anonymous — an ephemeral ID only, not your identity
Encryption	End-to-end, same session keys as normal messages	Signed but not encrypted (must be readable by any nearby responder); signature proves it isn't spoofed or duplicated
Opt-in	On by default once you've added emergency contacts	Explicit separate toggle, off by default

14.2 Flow

- User taps 'Need Help.' App asks which tier(s) to send — circle only, broadcast, or both.
- LocationManager captures a single current location fix (not continuous tracking).
- Tier 1 payload is encrypted with existing session/group keys and sent like a normal message, but flagged priority so it jumps the relay queue ahead of routine chat traffic and gets a longer TTL.
- Tier 2 payload is stripped of identity, location is rounded, signed with an ephemeral key, and flood-relayed the same way an admin broadcast is (Section 11) — any device shows it as a 'help needed' pin on the resource map.
- The request auto-expires after a set window (e.g. 24 hours) unless the user renews it, and can be cancelled manually at any time, which sends a follow-up 'resolved' signal on the same channel(s).

14.3 Core Functions / Module: HelpRequestManager

```
LocationManager.getCurrentLocation() -> { lat, lng, accuracy }
HelpRequestManager.triggerHelpRequest(tiers: ['circle','broadcast'], urgencyType)
HelpRequestManager.cancelHelpRequest(requestId) -> sends signed 'resolved' follow-up
HelpRequestManager.roundToApproxGrid(lat, lng) -> anonymized coordinates for Tier 2
MapUI.listActiveHelpPins() -> [ {approxLocation, urgencyType, timestamp, expiresAt} ]
```

14.4 New Functional Requirements

ID	Requirement	Priority
FR-29	Location shall be captured only at the moment of an explicit user trigger, never passively.	High
FR-30	Tier 1 requests shall be end-to-end encrypted to the user's designated emergency contacts.	High
FR-31	Tier 2 broadcasts shall strip personal identity and round location before transmission.	High
FR-32	Help requests shall carry relay priority (shorter queue wait, longer TTL) over routine messages.	Medium
FR-33	Help requests shall auto-expire and support manual cancellation with a 'resolved' follow-up.	Medium

15. Sign-Off

Reviewed By	Signature	Date
Product Owner		
Technical Lead		